JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class DelayQueue<E extends Delayed>

java.lang.Object
  java.util.AbstractCollection<E>
    java.util.AbstractQueue<E>
      java.util.concurrent.DelayQueue<E>

Type Parameters:

E - the type of elements held in this queue

All Implemented Interfaces:

Iterable<E>, Collection<E>, BlockingQueue<E>, Queue<E>

```
public class DelayQueue<E extends Delayed>
extends AbstractQueue<E>
implements BlockingQueue<E>
```

An unbounded blocking queue of Delayed elements, in which an element generally becomes eligible for removal when its delay has expired.

An element is considered *expired* when its `getDelay(TimeUnit.NANOSECONDS)` method would return a value less than or equal to zero.

An element is considered the *head* of the queue if it is the element with the earliest expiration time, whether in the past or the future, if there is such an element.

An element is considered the *expired head* of the queue if it is the *expired* element with the earliest expiration time in the past, if there is such an element. The *expired head*, when present, is also the *head*.

While this class implements the `BlockingQueue` interface, it intentionally violates the general contract of `BlockingQueue`, in that the following methods disregard the presence of unexpired elements and only ever remove the *expired head*:

- `poll()`
- `poll(long,TimeUnit)`
- `take()`
- `remove()`

All other methods operate on both expired and unexpired elements. For example, the `size()` method returns the count of all elements. Method `peek()` may return the (non-null) *head* even when `take()` would block waiting for that element to expire.

This queue does not permit null elements.

This class and its iterator implement all of the *optional* methods of the `Collection` and `Iterator` interfaces. The Iterator provided in method `iterator()` is *not* guaranteed to traverse the elements of the DelayQueue in any particular order.

This class is a member of the Java Collections Framework.

Since:

1.5

Constructor Summary

Constructors

Constructor	Description
`DelayQueue()`	Creates a new `DelayQueue` that is initially empty.
`DelayQueue(Collection<? extends E> c)`	Creates a `DelayQueue` initially containing the elements of the given collection of `Delayed` instances.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
`boolean`	`add(E e)`	Inserts the specified element into this delay queue.
`void`	`clear()`	Atomically removes all of the elements from this delay queue.
`int`	`drainTo(Collection<? super E> c)`	Removes all available elements from this queue and adds them to the given collection.
`int`	`drainTo(Collection<? super E> c, int maxElements)`	Removes at most the given number of available elements from this queue and adds

		them to the given collection.
Iterator<E>	**iterator**()	Returns an iterator over all the elements (both expired and unexpired) in this queue.
boolean	**offer**(E e)	Inserts the specified element into this delay queue.
boolean	**offer**(E e, long timeout, TimeUnit unit)	Inserts the specified element into this delay queue.
E	**peek**()	Retrieves, but does not remove, the head of this queue, or returns null if this queue is empty.
E	**poll**()	Retrieves and removes the expired head of this queue, or returns null if this queue has no expired elements.
E	**poll**(long timeout, TimeUnit unit)	Retrieves and removes the expired head of this queue, waiting if necessary until an expired element is available on this queue, or the specified wait time expires.
void	**put**(E e)	Inserts the specified element into this delay queue.
int	**remainingCapacity**()	Always returns Integer.MAX_VALUE

		because a DelayQueue is not capacity constrained.
E	**remove**()	Retrieves and removes the expired head of this queue, or throws an exception if this queue has no expired elements.
boolean	**remove**(Object o)	Removes a single instance of the specified element from this queue, if it is present, whether or not it has expired.
int	**size**()	Returns the number of elements in this collection.
E	**take**()	Retrieves and removes the expired head of this queue, waiting if necessary until an expired element is available on this queue.
Object[]	**toArray**()	Returns an array containing all of the elements in this queue.
<T> T[]	**toArray**(T[] a)	Returns an array containing all of the elements in this queue; the runtime type of the returned array is that of the specified array.

Methods declared in class AbstractQueue

addAll, element

Methods declared in class AbstractCollection

`contains`, `containsAll`, `isEmpty`, `removeAll`, `retainAll`, `toString`

Methods declared in class Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `wait`, `wait`, `wait`

Methods declared in interface BlockingQueue

`contains`

Methods declared in interface Collection

`equals`, `hashCode`, `parallelStream`, `removeIf`, `spliterator`, `stream`, `toArray`

Methods declared in interface Iterable

`forEach`

Constructor Details

DelayQueue

```
public DelayQueue()
```

Creates a new `DelayQueue` that is initially empty.

DelayQueue

```
public DelayQueue(Collection<? extends E> c)
```

Creates a `DelayQueue` initially containing the elements of the given collection of `Delayed` instances.

Parameters:

`c` - the collection of elements to initially contain

Throws:

`NullPointerException` - if the specified collection or any of its elements are null

Method Details

add

`public boolean add(E e)`

Inserts the specified element into this delay queue.

Specified by:

`add` in interface `BlockingQueue<E extends Delayed>`

Specified by:

`add` in interface `Collection<E extends Delayed>`

Specified by:

`add` in interface `Queue<E extends Delayed>`

Overrides:

`add` in class `AbstractQueue<E extends Delayed>`

Parameters:

e - the element to add

Returns:

`true` (as specified by `Collection.add(E)`)

Throws:

`NullPointerException` - if the specified element is null

offer

`public boolean offer(E e)`

Inserts the specified element into this delay queue.

Specified by:

`offer` in interface `BlockingQueue<E extends Delayed>`

Specified by:

`offer` in interface `Queue<E extends Delayed>`

Parameters:

e - the element to add

Returns:

`true`

Throws:

`NullPointerException` - if the specified element is null

put

```
public void put(E e)
```

Inserts the specified element into this delay queue. As the queue is unbounded this method will never block.

Specified by:

put in interface BlockingQueue<E extends Delayed>

Parameters:

e - the element to add

Throws:

NullPointerException - if the specified element is null

offer

```
public boolean offer(E e,
                     long timeout,
                     TimeUnit unit)
```

Inserts the specified element into this delay queue. As the queue is unbounded this method will never block.

Specified by:

offer in interface BlockingQueue<E extends Delayed>

Parameters:

e - the element to add

timeout - This parameter is ignored as the method never blocks

unit - This parameter is ignored as the method never blocks

Returns:

true

Throws:

NullPointerException - if the specified element is null

poll

```
public E poll()
```

Retrieves and removes the expired head of this queue, or returns null if this queue has no expired elements.

Specified by:

poll in interface Queue<E extends Delayed>

Returns:

the *expired head* of this queue, or `null` if this queue has no elements with an expired delay

take

```
public E take()
       throws InterruptedException
```

Retrieves and removes the expired head of this queue, waiting if necessary until an expired element is available on this queue.

Specified by:

`take` in interface `BlockingQueue<E extends Delayed>`

Returns:

the *expired head* of this queue

Throws:

`InterruptedException` - if interrupted while waiting

poll

```
public E poll(long timeout,
              TimeUnit unit)
       throws InterruptedException
```

Retrieves and removes the expired head of this queue, waiting if necessary until an expired element is available on this queue, or the specified wait time expires.

Specified by:

`poll` in interface `BlockingQueue<E extends Delayed>`

Parameters:

`timeout` - how long to wait before giving up, in units of `unit`

`unit` - a `TimeUnit` determining how to interpret the `timeout` parameter

Returns:

the *expired head* of this queue, or `null` if the specified waiting time elapses before an element with an expired delay becomes available

Throws:

`InterruptedException` - if interrupted while waiting

remove

```
public E remove()
```

Retrieves and removes the expired head of this queue, or throws an exception if this queue has no expired elements.

Specified by:

`remove` in interface `Queue<E extends Delayed>`

Overrides:

`remove` in class `AbstractQueue<E extends Delayed>`

Returns:

the *expired head* of this queue

Throws:

`NoSuchElementException` - if this queue has no elements with an expired delay

Since:

21

peek

```
public E peek()
```

Retrieves, but does not remove, the head of this queue, or returns `null` if this queue is empty. Unlike `poll`, if no expired elements are available in the queue, this method returns the element that will expire next, if one exists.

Specified by:

`peek` in interface `Queue<E extends Delayed>`

Returns:

the *head* of this queue, or `null` if this queue is empty

size

```
public int size()
```

Description copied from interface: `Collection`

Returns the number of elements in this collection. If this collection contains more than `Integer.MAX_VALUE` elements, returns `Integer.MAX_VALUE`.

Specified by:

`size` in interface `Collection<E extends Delayed>`

Returns:

the number of elements in this collection

drainTo

```
public int drainTo(Collection<? super E> c)
```

Description copied from interface: `BlockingQueue`

Removes all available elements from this queue and adds them to the given collection. This operation may be more efficient than repeatedly polling this queue. A failure encountered while attempting to add elements to collection `c` may result in elements being in neither, either or both collections when the associated exception is thrown. Attempts to drain a queue to itself result in `IllegalArgumentException`. Further, the behavior of this operation is undefined if the specified collection is modified while the operation is in progress.

Specified by:

`drainTo` in interface `BlockingQueue<E extends Delayed>`

Parameters:

`c` - the collection to transfer elements into

Returns:

the number of elements transferred

Throws:

`UnsupportedOperationException` - if addition of elements is not supported by the specified collection

`ClassCastException` - if the class of an element of this queue prevents it from being added to the specified collection

`NullPointerException` - if the specified collection is null

`IllegalArgumentException` - if the specified collection is this queue, or some property of an element of this queue prevents it from being added to the specified collection

drainTo

```
public int drainTo(Collection<? super E> c,
                   int maxElements)
```

Description copied from interface: `BlockingQueue`

Removes at most the given number of available elements from this queue and adds them to the given collection. A failure encountered while attempting to add elements to collection `c` may result in elements being in neither, either or both collections when the associated exception is thrown. Attempts to drain a queue to itself result in `IllegalArgumentException`. Further, the behavior of this operation is undefined if the specified collection is modified while the operation is in progress.

Specified by:

`drainTo` in interface `BlockingQueue<E extends Delayed>`

Parameters:

`c` - the collection to transfer elements into

`maxElements` - the maximum number of elements to transfer

Returns:

the number of elements transferred

Throws:

`UnsupportedOperationException` - if addition of elements is not supported by the specified collection

`ClassCastException` - if the class of an element of this queue prevents it from being added to the specified collection

`NullPointerException` - if the specified collection is null

`IllegalArgumentException` - if the specified collection is this queue, or some property of an element of this queue prevents it from being added to the specified collection

clear

```
public void clear()
```

Atomically removes all of the elements from this delay queue. The queue will be empty after this call returns. Elements with an unexpired delay are not waited for; they are simply discarded from the queue.

Specified by:

`clear` in interface `Collection<E extends Delayed>`

Overrides:

`clear` in class `AbstractQueue<E extends Delayed>`

remainingCapacity

```
public int remainingCapacity()
```

Always returns `Integer.MAX_VALUE` because a `DelayQueue` is not capacity constrained.

Specified by:

`remainingCapacity` in interface `BlockingQueue<E extends Delayed>`

Returns:

`Integer.MAX_VALUE`

toArray

```
public Object[] toArray()
```

Returns an array containing all of the elements in this queue. The returned array elements are in no particular order.

The returned array will be "safe" in that no references to it are maintained by this queue. (In other words, this method must allocate a new array). The caller is thus free to modify the returned array.

This method acts as bridge between array-based and collection-based APIs.

Specified by:

`toArray` in interface `Collection<E extends Delayed>`

Overrides:

`toArray` in class `AbstractCollection<E extends Delayed>`

Returns:

an array containing all of the elements in this queue

toArray

```
public <T> T[] toArray(T[] a)
```

Returns an array containing all of the elements in this queue; the runtime type of the returned array is that of the specified array. The returned array elements are in no particular order. If the queue fits in the specified array, it is returned therein. Otherwise, a new array is allocated with the runtime type of the specified array and the size of this queue.

If this queue fits in the specified array with room to spare (i.e., the array has more elements than this queue), the element in the array immediately following the end of the queue is set to `null`.

Like the `toArray()` method, this method acts as bridge between array-based and collection-based APIs. Further, this method allows precise control over the runtime type of the output array, and may, under certain circumstances, be used to save allocation costs.

The following code can be used to dump a delay queue into a newly allocated array of `Delayed`:

```
Delayed[] a = q.toArray(new Delayed[0]);
```

Note that `toArray(new Object[0])` is identical in function to `toArray()`.

Specified by:

`toArray` in interface `Collection<E extends Delayed>`

Overrides:

`toArray` in class `AbstractCollection<E extends Delayed>`

Type Parameters:

T - the component type of the array to contain the collection

Parameters:

a - the array into which the elements of the queue are to be stored, if it is big enough; otherwise, a new array of the same runtime type is allocated for this purpose

Returns:

an array containing all of the elements in this queue

Throws:

`ArrayStoreException` - if the runtime type of the specified array is not a supertype of the runtime type of every element in this queue

`NullPointerException` - if the specified array is null

remove

```
public boolean remove(Object o)
```

Removes a single instance of the specified element from this queue, if it is present, whether or not it has expired.

Specified by:

`remove` in interface `BlockingQueue<E extends Delayed>`

Specified by:

`remove` in interface `Collection<E extends Delayed>`

Overrides:

`remove` in class `AbstractCollection<E extends Delayed>`

Parameters:

o - element to be removed from this collection, if present

Returns:

`true` if an element was removed as a result of this call

iterator

```
public Iterator<E> iterator()
```

Returns an iterator over all the elements (both expired and unexpired) in this queue. The iterator does not return the elements in any particular order.

The returned iterator is *weakly consistent*.

Specified by:

`iterator` in interface `Collection<E extends Delayed>`

Specified by:

`iterator` in interface `Iterable<E extends Delayed>`

Specified by:

`iterator` in class `AbstractCollection<E extends Delayed>`

Returns:

an iterator over the elements in this queue

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.